

Warehouse Design Issues

 Uploaded image

explain this in details

Sure! The image provides a summary of two key concepts in data warehousing:

More Warehouse Design Issues

1. Data Transformation and Data Cleansing

- **Data transformation** is the process of converting data into a suitable format for analysis and storage in the data warehouse.
- **Data cleansing** refers to correcting errors and inconsistencies in data. For example:
 - Fixing misspellings or formatting errors in addresses (e.g., "New Yrok" → "New York")
 - Resolving ZIP code issues
 - Merging multiple address lists from different data sources
 - Removing duplicate records to ensure data consistency

2. How to Propagate Updates

- Since a data warehouse often integrates data from multiple sources, any update in the source systems needs to be reflected in the warehouse.
- The warehouse schema can be:
 - A **materialized view**, meaning a snapshot of data from the source is stored in the warehouse.
 - Or a **virtual view**, where data is fetched live from the source.

- **View maintenance** refers to the process of keeping these views up to date when the source data changes.

3. What Data to Summarize

- Raw data is typically very large and can be too expensive or slow to keep fully online.
- Instead, **aggregate values** (like totals and subtotals) can be stored, which are often enough for decision-making.
- Queries on detailed (raw) data can be optimized by using these precomputed aggregates, reducing response times and improving performance.

Multidimensional Data and Warehouse Schemas

This section discusses how data is structured in data warehouses using **fact** and **dimension** tables in a multidimensional schema.

1. Fact Tables

- These are the central tables in a data warehouse and contain the **measurable, quantitative data**.
- Fact tables are **large** because they store transactional data (e.g., sales records).
- Example: A `sales` table with fields:

```
bash
```

```
item_id, store_id, customer_id, date, number, price
```

2. Dimension Tables

- These are **smaller tables** that provide **context or descriptive information** related to the data in the fact tables.
- Example: A `store` dimension table might contain store name, address, region, etc.

3. Attributes of Fact Tables

- Fact tables contain two types of attributes:

a. Measure Attributes

- These are **quantitative values** we care about, which can be aggregated (e.g., sum, average).
- Example: `number` (items sold) or `price` in the sales relation.

b. Dimension Attributes

- These are **qualitative values** that describe the "dimensions" along which data is analyzed.
 - Example: `item_id`, `color`, `store_id`, or `date`.
 - These are usually **foreign keys** that link to dimension tables.
-

Summary:

A data warehouse stores large volumes of data for analytical processing. It requires:

- Cleaning and transforming incoming data,
- Deciding what data (or summaries) to store,
- Structuring the data in a **multidimensional schema** with:
 - Large **fact tables** for transactional data,
 - Smaller **dimension tables** for descriptive context.

Let me know if you want a diagram or example to visualize this better!

 Uploaded image

explain this

This slide explains **how databases support data warehouses**, focusing on performance optimization techniques suited for analytics-heavy environments. Here's a detailed breakdown:

1. Data in Warehouses is Usually Append-Only

- In a **typical data warehouse**, data is **not updated** or deleted often. Instead, **new data is appended** (added).
 - This simplifies the system because:
 - There's **no need for concurrency control**, which is usually necessary in OLTP (Online Transaction Processing) systems to manage simultaneous updates.
 - Reduces complexity and improves performance for bulk insert operations.
-

2. Column-Oriented Storage

- **Traditional databases** use **row-oriented storage** (all values of a single record are stored together).
- **Data warehouses**, on the other hand, often use **column-oriented storage**, where values of each column are stored separately.
 - For example, in a `sales` table:
 - All values of `item_id` are stored as one array.
 - All values of `store_id` are stored as another array.
 - All values of `date`, `price`, `quantity`, etc. are stored similarly.

Advantages of Column-Oriented Storage:

- **Efficient querying:** If a user only wants `price` and `item_id`, the system doesn't have to load entire rows — just the needed columns.
 - **Reduces I/O and memory usage:** Only relevant data is fetched into memory.
 - Ideal for **analytical queries** that often aggregate data from a few columns over many rows (e.g., calculating total sales).
-

3. Parallel Storage and Query Processing Infrastructure

- Data warehouses handle **huge volumes of data**, so they often rely on **parallel processing**.
- This involves:
 - **Distributed file systems**: Store data across many machines.
 - **Parallel frameworks** like:
 - **MapReduce**: A programming model for processing large data sets across clusters.
 - **Hive**: A data warehouse tool built on Hadoop that lets users write SQL-like queries for big data.

Benefits:

- Faster data loading and querying by **distributing the workload**.
 - Scales well for **big data environments**.
-

Summary:

- Data warehouses are optimized for **efficient reading and analytics**.
- They avoid updates, use **column-based storage** to reduce unnecessary data loading, and leverage **parallel and distributed computing** to handle large-scale data efficiently.

Let me know if you want real-world examples or comparisons with OLTP systems!